C Programming In Space & Defense System

Mission-Critical Software

Subject name Foundation of Programming

Team members

- Sarang
- Ashutosh Kumar Dewangan
- Tanish Dange
- Sai Shirke
- Pranav Phuke

C in Embedded & Aerospace Systems

C is foundational for flight computers, satellites, missile guidance, and radar systems in aerospace and defense sectors.

Flight Computers

Real-time control for navigation, attitude, and propulsion.

Satellite Systems

Onboard processors for comms, telemetry, and payload operations.

Missile Guidance

Precision control algorithms for deterministic timing and reliability.

Radar Systems

Signal processing and tracking software for defense.

C excels in low-level hardware control, real-time tasks with deterministic timing, and efficient operation on resource-constrained microcontrollers.

Why C Still Dominates Aerospace & Defense

Performance & Efficiency

Near-hardware speed for real-time processing on limited resources.

Direct Hardware Access

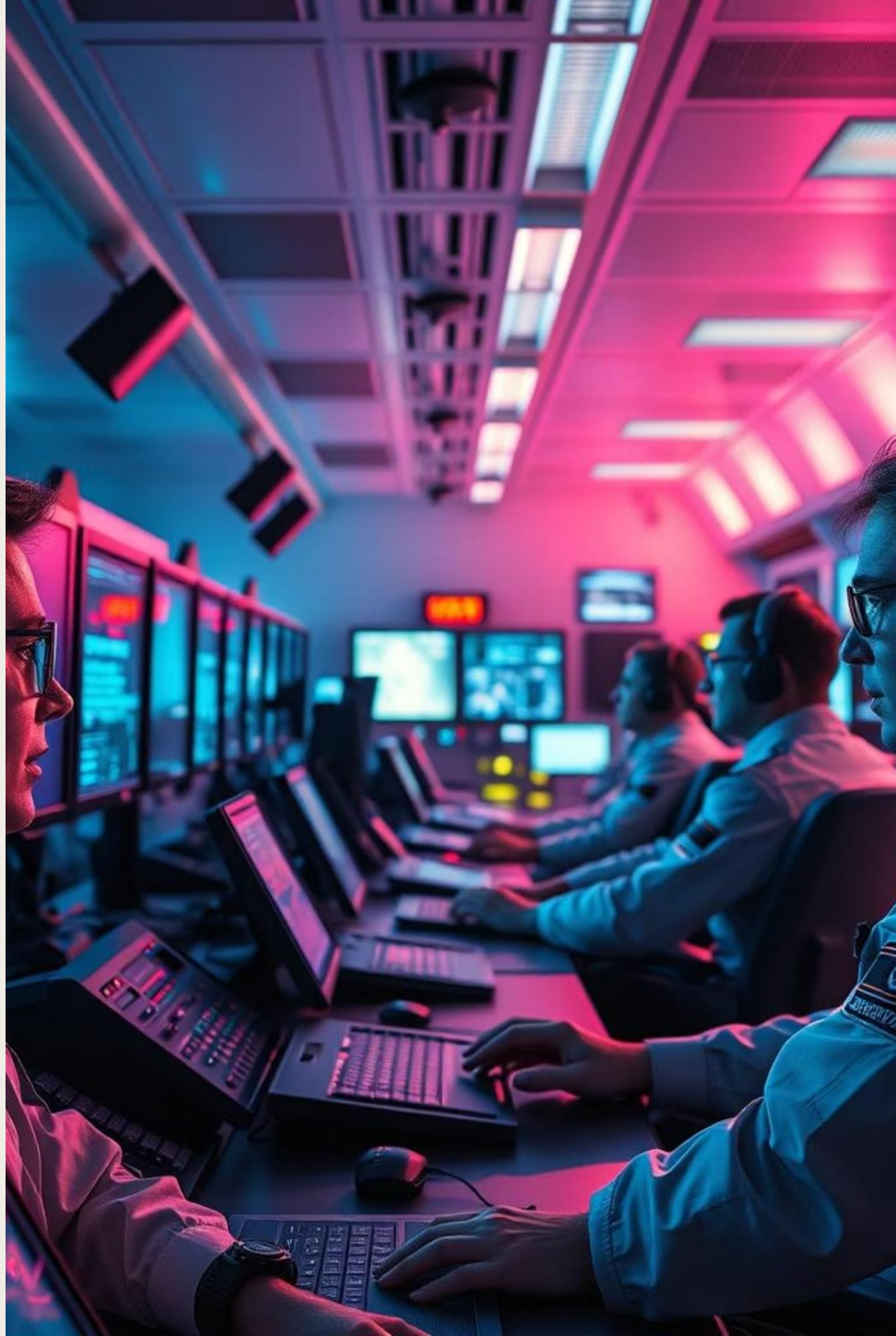
Direct control over hardware and system resources.

Mature Ecosystem

Established tools reduce development risk and accelerate certification.

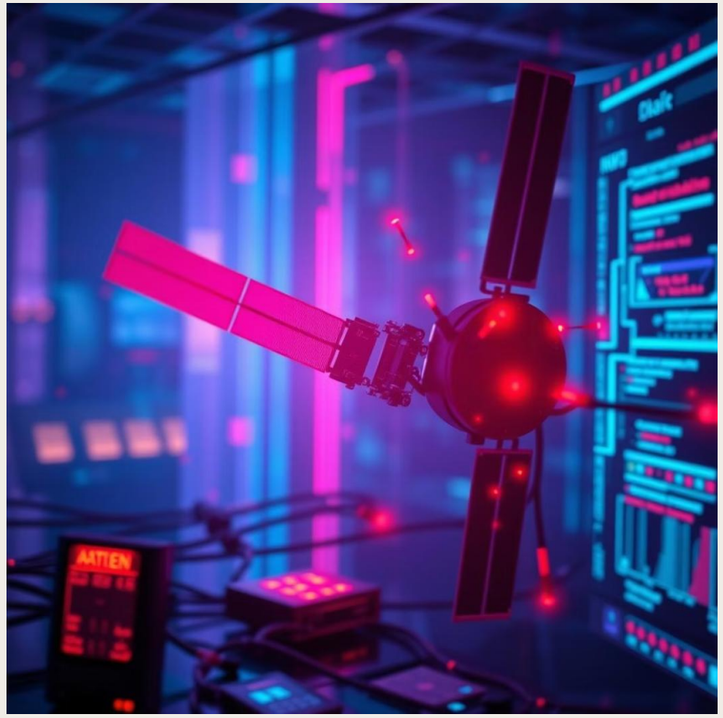
Legacy Codebase

Proven reliability from decades of critical mission software.



Memory Safety Challenges in C

Manual memory management in C creates vulnerabilities that can compromise mission-critical systems.



Buffer Overflows

Unchecked array access corrupts memory, leading to crashes or malicious code execution.

Pointer Mismanagement

Dangling pointers and use-after-free errors cause unpredictable system behavior.

Undefined Behavior

Gaps in language specification allow compiler optimizations to break safety-critical code assumptions.

Hidden Vulnerabilities

Rust rewrites frequently expose C's hidden memory safety bugs.

Cybersecurity Risks in Space Systems

28

Satellites Tracked

ESA tracks 28 satellites

42B

Monthly Events

Billions of potential attacks processed

3x

Attack Growth

Satellite cyber threats tripling

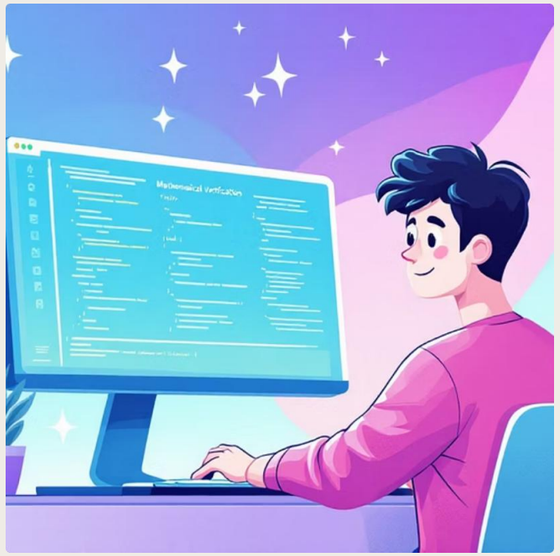
Space systems face increasing cyberattacks, exploiting C-based vulnerabilities. Successful exploits can disrupt GPS, communications, defense systems, and critical infrastructure. The growing number of connected satellites exponentially expands the attack surface.

Emerging Safer Approaches & Industry Trends

Aerospace & defense adopt modern languages and verification for C's memory safety, maintaining performance.

Rust Adoption

Rust's ownership model enables memory-safe aerospace components, eliminating vulnerabilities while matching C performance.

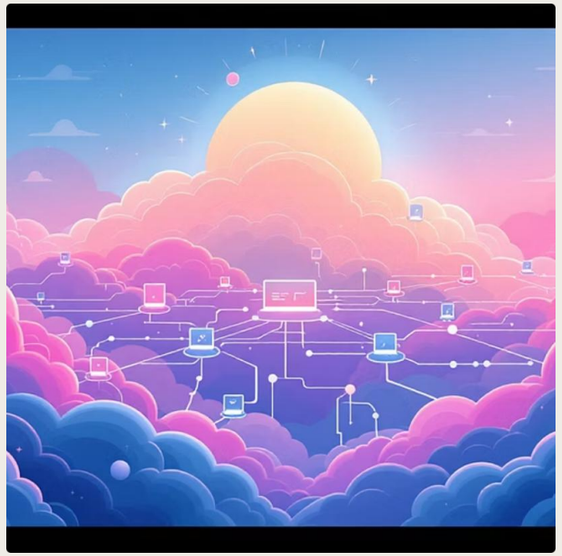


AdaCore Tools

Enhanced static analysis and formal verification prove correctness pre-deployment, reducing runtime defects.

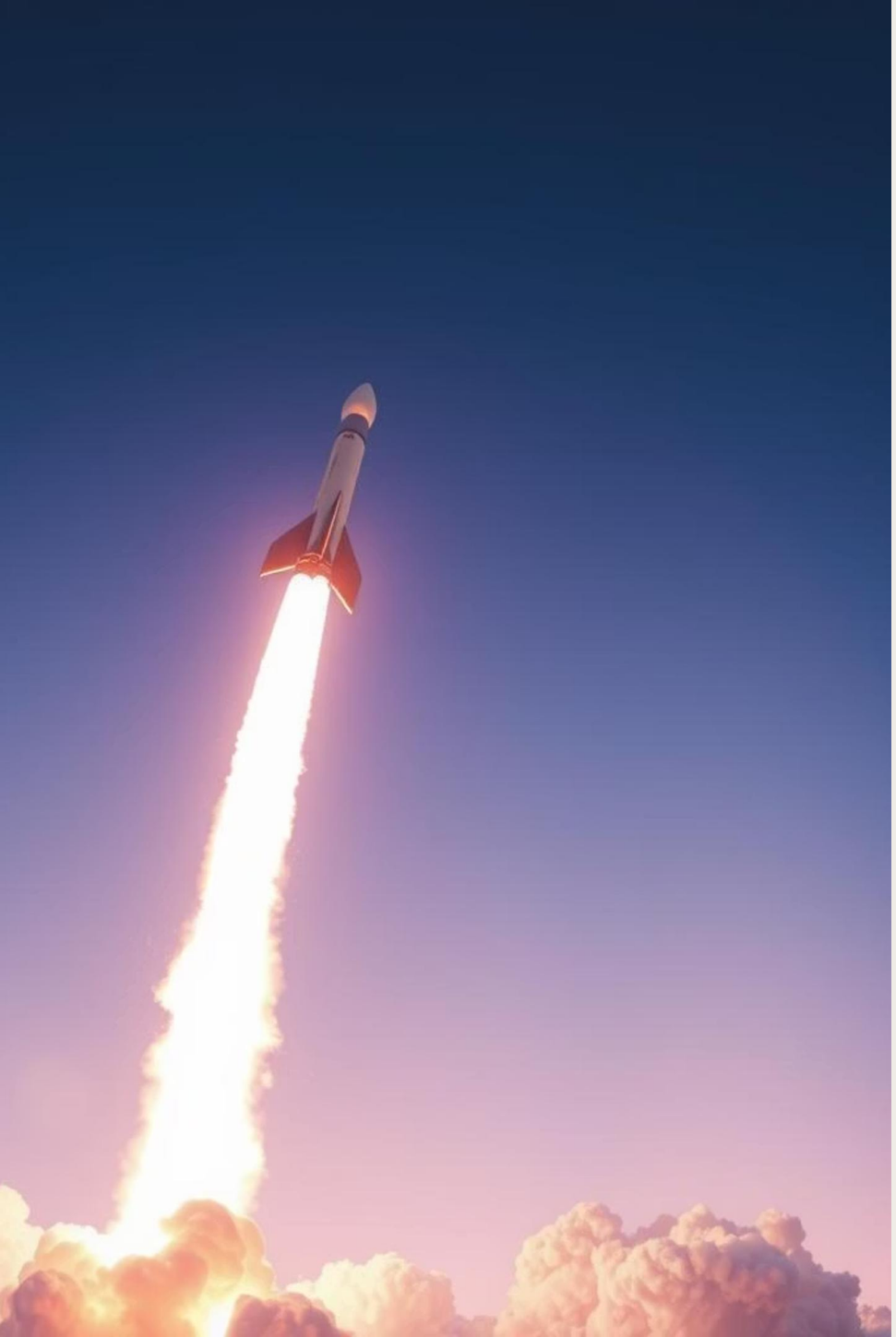
Memory Safety Extensions

Research into C memory safety allows gradual migration of legacy codebases without complete rewrites.



Hybrid Systems

Combining C modules with safer languages for new components balances reliability with modern security.



Conclusion: The Future of Mission-Critical Software

C Remains Foundational

Established codebases and toolchains power critical aerospace and defense missions.

Safety Requires Discipline

Memory safety and cybersecurity demand rigorous coding and verification.

Modern Practices Integrate

C performance with safer languages and formal methods creates next-gen systems.

Reliable & Secure Systems

Goal: Mission-critical software with proven reliability and modern security.